MASTER DIRECTIVE: BOSS EXECUTIVE INTELLIGENCE - COMPLETE APPLICATION AUDIT, GAP ANALYSIS, CONTROLLED IMPLEMENTATION AND FINAL VERIFICATION

1. PROJECT CONTEXT AND PRIMARY OBJECTIVE

The project is BOSS - Business Operating Strategic System.

The application has already been developed in this project using Claude Code. Your task is to audit, validate, improve and complete the existing application against the requirements in this document.

This is NOT a greenfield development project.

Do not rebuild the application, replace its architecture, recreate working features, or redesign the entire interface unnecessarily.

The objective is to make the existing BOSS application a reliable, evidence-based, AI-supported executive intelligence and decision-support system.

BOSS should help executives understand:

1. The organisation's current health.
2. Its present and projected financial resilience.
3. The health of six business dimensions.
4. The causes of current and emerging problems.
5. The actions management should prioritise.
6. Who is accountable for each action and whether the action produces the intended outcome.

The product promise is:

BOSS: One View. Complete Clarity. Smarter Decisions.

BOSS should connect verified business data to current health, future exposure, prioritised action, accountability and measured outcomes.

Your responsibilities are to:

- Discover the existing technology stack and architecture.
- Audit every requirement below against the actual codebase.
- Establish what is implemented, incomplete, missing, defective or unverified.
- Preserve existing working functionality.
- Identify and explain implementation gaps.
- Produce a complete requirements traceability register.
- Wait for approval before implementing changes.
- Implement approved gaps incrementally.

- Verify the implementation using actual evidence and appropriate tests.
- Reconcile every original requirement against the final code and test results.

Never claim that a feature is implemented, tested, accurate, secure or production-ready without sufficient evidence.

---

2. NON-NEGOTIABLE EXECUTION SEQUENCE

Follow these phases in order. Do not skip phases.

Phase 0 - Protect the existing application

Before changing anything:

1. Identify the project root and current working directory.
2. Inspect Git status, existing modifications, untracked files and available branches.
3. Identify the current application version and existing backup or rollback mechanisms.
4. Determine whether the working tree contains changes that must be preserved.
5. Identify any potentially destructive operations in your proposed workflow.
6. Avoid exposing secrets, credentials, private keys, production data or personal information.

Do not overwrite uncommitted work.

If the project is not under version control, recommend a safe backup procedure before any modification.

Do not create a backup by blindly copying sensitive files, dependency directories, large data stores or production credentials.

Do not install packages, modify files, run migrations, alter databases, change deployment configuration or access production services during the initial audit.

Phase 1 - Discover the actual technology stack

Inspect the existing source code, documentation and configuration.

Identify:

- Frontend framework and application entry points.
- Backend framework and service architecture.
- Programming languages and major dependencies.
- Database technology, schema, migrations and data-access patterns.
- APIs, integrations and external services.
- Authentication, authorisation, roles and permissions.
- Existing AI models, prompts, tools and orchestration logic.
- Financial calculations and business rules.
- Existing dashboards, reporting modules and workflows.
- Testing frameworks, test coverage and available test commands.
- Build, linting, type-checking and security-checking tools.
- Deployment configuration, environment variables and CI/CD.
- Existing logging, monitoring, audit trails and error handling.

Do not assume that a dependency is used merely because it appears in a package manifest.

Identify the actual implementation paths and how the major components interact.

If information cannot be established from the repository, report the limitation instead of guessing.

Phase 2 - Conduct the complete read-only requirements audit

Audit every requirement in Sections 4–15 below.

Do not change code during this phase.

Inspect the actual implementation, not merely documentation, comments, screen labels, database column names or the presence of a function.

Trace relevant workflows across frontend, backend, database, APIs, permissions, integrations and tests wherever applicable.

Produce the initial audit report and requirements register.

Then STOP and wait for my explicit approval before implementing any changes.

Phase 3 - Agree on the gap-remediation plan

After presenting the audit, identify:

- Existing features that are adequate and should remain unchanged.
- Existing features that require correction or extension.
- Missing features.
- Features that exist but are not sufficiently tested.
- Requirements that need business clarification.
- Dependencies, conflicts, risks and proposed implementation order.
- Any changes requiring architectural, financial, security or governance decisions.

Do not implement unresolved, materially ambiguous requirements by guessing.

Wait for approval of the proposed changes.

Phase 4 - Implement approved changes incrementally

Implement only the approved requirements.

Use small, coherent changes that fit the existing architecture.

Preserve existing working functionality and update the traceability register as work progresses.

Phase 5 - Test and independently reconcile the result

Run appropriate tests, inspect the actual diff, review regressions, and reconcile the complete original requirements list against the final implementation.

Do not mark the entire project complete if any mandatory requirement remains incomplete, blocked, failed or unverified.

---

3. REQUIREMENTS TRACEABILITY AND EVIDENCE STANDARD

Assign every requirement a permanent, unique identifier.

Use the following ID families where appropriate:

- ARCH: Architecture and application integrity.
- EXEC: Executive overview and experience.
- HLTH: Health score and status model.
- DIM: Six business dimensions.
- CASH: Cash runway and forecasting.
- DATA: Data quality and integrity.
- INTEL: AI intelligence and root-cause analysis.
- ACT: Recommendations and action management.
- GOV: Governance, traceability and controls.
- TEST: Testing and validation.
- OPS: Deployment, monitoring and ongoing operations.

For every requirement, record:

1. Requirement ID.
2. Requirement description.
3. Business rationale.
4. Priority and reason.
5. Acceptance criteria.
6. Existing implementation status.
7. Evidence from the current code.
8. Relevant file paths, functions, classes, APIs and database structures.
9. Missing or defective functionality.
10. Proposed remediation.
11. Dependencies and risks.
12. Verification procedure.
13. Actual test results.
14. Final implementation and verification status.

Use these statuses consistently:

- NOT AUDITED.
- IMPLEMENTED BUT NOT VERIFIED.
- PARTIALLY IMPLEMENTED.
- NOT IMPLEMENTED.
- IN PROGRESS.
- BLOCKED.
- VERIFIED.
- NOT APPLICABLE, WITH JUSTIFICATION.

Distinguish implementation status from verification status.

A source-code inspection is not the same as a successful runtime test. A passing unit test is not necessarily proof of end-to-end functionality.

For a requirement to be VERIFIED, its defined acceptance criteria must be met and the appropriate verification evidence must exist.

For each requirement, provide a clear evidence trail, including file paths and relevant line numbers when available.

Do not fabricate line numbers, test results, data sources or implementation evidence.

If a test cannot be executed, state why, what remains uncertain and what is needed to complete verification.

Create a permanent project file named REQUIREMENTS_TRACEABILITY.md only after the initial audit report has been presented and I approve creating the register.

Keep the register current throughout implementation.

Never silently remove, weaken, merge away or ignore an original requirement.

---

4. EXECUTIVE INTELLIGENCE EXPERIENCE

BOSS must be evaluated as an integrated executive decision-support experience, not simply as a dashboard of isolated metrics.

Audit whether the existing application presents the following six intelligence layers on a consolidated executive page, with drill-down access to supporting details.

Layer 1 - Overall business health

Answer: How healthy is the organisation today?

Show the overall health score, final business status, major contributing factors, significant changes and any active critical-risk override.

Layer 2 - Cashflow and runway

Answer: Can the organisation sustain its obligations and current trajectory?

Show usable cash, reporting date, forecast-based runway, relevant minimum operating cash threshold, projected liquidity breach, and expected, optimistic and downside scenarios.

Layer 3 - Functional health

Answer: Which of the six business dimensions are healthy, deteriorating or critical?

Show Finance, Accounting, Auditing, Compliance, Risk and Human Resources, including relevant scores, status, trends, data confidence and material exceptions.

Layer 4 - Intelligence and root causes

Answer: Why is the current position occurring, and what is likely to happen next?

Connect relevant data across dimensions, distinguish facts from hypotheses, explain material drivers and quantify forecast impacts where supported by data.

Layer 5 - Prioritised action plan

Answer: What must leadership do first, why, by when and with what expected impact?

Display evidence-backed actions ranked by urgency, materiality, likelihood, time sensitivity and expected benefit.

Layer 6 - Accountability and outcomes

Answer: Who owns each action, what progress has been made, and did it improve the result?

Track accountable owners, deadlines, status, completion evidence, expected outcomes, actual outcomes and relevant follow-up measurements.

Executive page requirements

Audit the existing executive overview for these sections, in this logical order:

1. Top bar: BOSS Intelligence, organisation selector, reporting period, reporting date, data freshness and refresh status.
2. Executive verdict: overall score, final status, changes since the previous assessment and critical-risk overrides.
3. Financial resilience: available cash, forecast-based runway, minimum cash buffer, projected breach date and scenario comparisons.
4. Six-dimension health: Finance, Accounting, Auditing, Compliance, Risk and Human Resources.
5. AI Intelligence Brief: material findings, verified evidence, root causes, cross-functional implications and uncertainty.
6. Prioritised action plan: priority, finding, evidence, action, impact, owner, deadline and status.
7. Confidence and traceability: data completeness, stale inputs, reconciliation issues, forecast confidence and supporting records.

Preserve the existing design system and functional components wherever possible.

Do not redesign the entire application simply to match this ordering. Identify gaps in the existing experience and propose the smallest effective UI changes.

Detailed departmental information should remain accessible through drill-downs rather than overwhelming the executive summary.

All displayed metrics must be grounded in actual organisation-specific data. Do not hardcode illustrative figures or pretend that a conceptual mock-up represents the current implementation.

---

5. HYBRID ORGANISATIONAL HEALTH MODEL

BOSS should maintain four connected but distinct outputs.

5.1 Overall health score

A score from 0 to 100 representing the weighted condition of the organisation's six business dimensions.

5.2 Business status

A plain-language Green, Amber or Red executive status.

5.3 Critical-risk override

A predefined and evidence-backed critical event must be capable of elevating the final business status, regardless of a high overall score.

For example, an otherwise strong score must not conceal an imminent inability to meet payroll or another verified critical obligation.

The score must remain visible for context.

The final executive status should reflect the most severe applicable verified condition, taking into account critical-risk rules, the cash outlook and material data-confidence limitations.

5.4 AI intelligence explanation

Explain:

- What the current position is.
- Why the position exists.
- Which factors contribute most to it.
- What has changed since the previous assessment.
- What may happen next if no action is taken.
- Which actions could improve the position.

The numerical score must be calculated by a deterministic, auditable calculation engine.

AI may interpret verified results and explain their implications. It must not silently invent, override or alter the underlying numerical calculations.

5.5 Proposed score bands

Audit the existing thresholds and determine how they should be configured.

The following are proposed initial BOSS thresholds, not validated universal standards:

- 85–100: Green - strong overall health; continue monitoring.
- 70–84: Green / Watch - generally healthy; address emerging weaknesses.
- 50–69: Amber - material weaknesses require management attention.
- 0–49: Red - significant deterioration or multiple severe weaknesses.

The precise presentation of Green / Watch within the Green-Amber-Red status model must be made consistent and unambiguous.

The score bands must be configurable and versioned. They must not be treated as validated industry benchmarks without supporting evidence.

The final status must not be determined by score bands alone.

A critical-risk override takes precedence over a favourable baseline score.

5.6 Six dimension weights

Audit the existing calculation logic against these proposed initial weights:

- Finance: 30%.
- Accounting: 15%.
- Auditing: 10%.
- Compliance: 15%.
- Risk: 20%.
- Human Resources: 10%.

Total: 100%.

These are starting hypotheses, not universal optimal weights.

Check whether weights are normalised, configurable, versioned, consistently applied and appropriate to the organisation.

Do not change established production weights without approval.

Where appropriate, support organisation-specific or industry-specific configurations, subject to defined governance.

5.7 Weighted score calculation

The baseline overall health score is:

$H = \Sigma(w_i \times S_i)$, for i = 1 to 6.

Where:

- H is the overall health score.
- S_i is the normalised score of dimension i, between 0 and 100.
- w_i is the weight of dimension i.
- The weights sum to 1.

An illustrative set of dimension scores is:

- Finance: 82.
- Accounting: 88.
- Auditing: 71.
- Compliance: 91.
- Risk: 63.
- Human Resources: 69.

Using the proposed weights, the baseline weighted score is 77.0.

This is an example for testing the formula, not an assertion about the organisation's actual health.

Check the arithmetic, numeric precision, rounding, validation of weight totals and handling of invalid inputs.

5.8 Dimension scoring

For each dimension, inspect:

- Indicator definitions and business meaning.
- Normalisation formulas.
- Thresholds and severity levels.
- Source data and reporting periods.
- Missing, stale and conflicting inputs.
- Weighting of individual indicators.
- Treatment of negative, out-of-range or anomalous values.
- Trend calculations and period-to-period comparisons.
- Versioning and reproducibility.

Do not invent universal scoring formulas for indicators whose business meaning and source data have not been established.

If the existing application lacks a defensible formula, document the gap and propose a configurable scoring specification for approval.

5.9 Critical-risk override engine

Inspect whether critical-risk conditions are implemented through explicit, testable business rules.

Candidate conditions include:

- Projected cash deficit before the next reliable funding source.
- Inability to meet critical payroll, debt or statutory payments.
- Material unresolved regulatory breach.
- Severe cybersecurity or operational incident threatening continuity.
- Material financial data integrity failures that invalidate the assessment.

These are candidate rules, not assumptions that every organisation uses identical thresholds.

Every configured override must specify:

- Rule ID and version.
- Severity.
- Evidence requirements.
- Source system and relevant records.
- Trigger threshold.
- Applicable time horizon.
- Affected business dimension.
- Status consequence.
- Responsible owner.
- Activation and resolution criteria.
- Audit history.

Distinguish confirmed critical events from unverified alerts, stale data and model-generated suspicions.

Where a critical risk cannot be confirmed because essential data is missing, do not falsely claim that the risk is absent. Use a clearly disclosed data-confidence warning and apply any approved conservative-status rule.

Ensure the final status cannot accidentally be reduced by a favourable average score when a confirmed critical override remains active.

5.10 Health score acceptance tests

Include tests for:

- All dimension scores equal to 100.
- All dimension scores equal to 0.
- Boundary values of score bands.
- Weights that do not sum to 1.
- Missing dimension scores.
- Invalid or out-of-range scores.
- A high baseline score with a critical-risk override.
- A low score without a critical-risk override.
- Multiple simultaneous critical risks.

- Override resolution and reassessment.
- Reproducibility using the same inputs and rule version.

Verify that a baseline score of 77 produces the expected numerical result under the proposed example weights and that a verified critical event can elevate the final status independently.

---

6. SIX BUSINESS DIMENSIONS

Audit the following six dimensions individually and as an integrated model.

Finance

Assess the relevant available financial information, including cash, cashflow, receivables, payables, profitability, expenses, working capital, forecast requirements and obligations where those data are available.

Accounting

Assess financial record completeness, reconciliations, ledger integrity, reporting accuracy, close processes, adjustments and other relevant accounting controls.

Auditing

Assess audit findings, control weaknesses, open observations, severity, remediation ownership, deadlines and overdue issues.

Compliance

Assess applicable obligations, deadlines, confirmed breaches, control failures and remediation status based on the organisation's configured regulatory and policy requirements.

Risk

Assess material enterprise, financial, operational, technology, cybersecurity, continuity and other configured risks.

Human Resources

Assess relevant workforce costs, capacity, productivity, staffing, retention, workforce risks and other approved business indicators.

Do not invent data for any dimension.

Do not assume that the absence of records proves that a dimension is healthy.

For each dimension, identify its authoritative sources, scoring specification, material exceptions, trends, data confidence and supporting evidence.

Where relevant, connect the dimensions without double-counting the same underlying event.

---

7. CASH RUNWAY AND FINANCIAL FORECASTING

Cash runway is a core BOSS capability and must be audited thoroughly.

7.1 Distinguish the key cash concepts

The implementation must distinguish:

- Current bank balances.
- Available unrestricted cash.
- Restricted or otherwise unavailable cash.
- Expected cash receipts.
- Expected cash payments.
- Committed funding.
- Uncommitted or contingent funding.
- Minimum operating cash threshold.
- Projected cash balances.
- Reference runway.
- Forecast-based runway.

Inspect whether the existing application uses the correct cash definitions and avoids counting the same balance or funding event more than once.

7.2 Reference runway

For a business experiencing positive net cash burn:

Reference runway in months = Available unrestricted cash / Average monthly net cash burn.

Net cash burn is cash outflows minus cash inflows over the selected period.

For example:

- Available unrestricted cash: ₹240 lakh.
- Average monthly net cash burn: ₹30 lakh.
- Reference runway: 8 months.

This is an illustrative calculation only.

Verify units, reporting periods, treatment of exceptional cash movements, and the validity of the underlying data.

Do not divide by zero or present misleading negative or infinite runway values.

7.3 Forecast-based runway

The primary runway measure should be derived from dated projected cash movements.

For each period t:

$C_t = C_{t-1} + I_t - O_t + F_t$

Where:

- C_t = projected closing cash balance.
- I_t = expected cash inflows.
- O_t = expected cash outflows.
- F_t = eligible financing or other funding flows.

The forecast must use a clearly defined opening cash position and consistent period boundaries.

Forecast-based runway is the time until projected usable cash falls below the configured minimum operating cash threshold.

Do not assume that zero cash is the appropriate threshold for every organisation.

Check the accuracy of the projected breach date, forecast horizon, timing conventions, cash availability assumptions and treatment of funding events.

7.4 Forecast scenarios

BOSS must support three clearly distinguished scenarios:

1. Optimistic.
2. Base case.
3. Downside.

Each scenario must use the same verified opening cash position and documented, scenario-specific assumptions.

Relevant assumptions may include:

- Customer collection timing.
- Sales and revenue.
- Gross margins.
- Operating expenses.
- Payroll and workforce costs.
- Working capital.
- Capital expenditure.
- Debt service.
- Tax and statutory payments.
- Funding timing and availability.

Do not simply multiply the base-case runway by arbitrary factors.

Use dated receipts and payments wherever possible.

Funding that is not sufficiently committed must not be treated as guaranteed funding in the committed-funding base case. Represent it separately as a contingent scenario or clearly identified assumption.

Display the assumptions that explain the divergence between scenarios.

7.5 Forecast horizons

Audit for two linked forecasting horizons:

- Short-term liquidity control: a rolling 13-week cash forecast, updated at least weekly where operationally appropriate.
- Strategic outlook: a rolling 12-month forecast, updated monthly or when material business conditions change.

These are proposed operating practices. Adapt the update cadence to the organisation's reporting maturity and cash volatility.

Check whether the existing application actually supports these horizons and update mechanisms.

Do not label a forecast as rolling if it does not automatically or operationally support the required period extension and refresh.

7.6 Runway display rules

Verify that BOSS handles the following cases correctly:

Positive net burn with usable cash: Show reference runway and forecast-based runway.

Positive operating cash generation: State that the business is cash-generative under current assumptions, when supported by evidence, and show forecast liquidity and downside exposure.

Zero or near-zero net burn: Do not display an arbitrary infinite runway. Show that reference runway is not meaningful under current burn conditions and present forecast-based liquidity.

Cash below the minimum operating threshold: Clearly report that the minimum liquidity threshold is breached.

Forecast threshold breach: Show the projected breach date, remaining time and relevant scenario.

No breach within the forecast horizon: State that runway is at least the forecast horizon, if justified, rather than claiming unlimited runway.

Unsecured funding: Exclude uncommitted funding from the committed-funding base case and show its effect separately.

Missing or stale data: Disclose that the forecast is incomplete, stale or not sufficiently reliable. Do not present unsupported precision.

Restricted cash: Exclude cash that cannot be used for the relevant obligations.

Negative projected balances: Do not conceal a forecast deficit by silently flooring the calculation at zero. If a chart needs a visual floor, preserve the actual calculated values and clearly label the presentation convention.

7.7 Cash forecasting tests

Test at least:

- Positive net burn.
- Positive operating cash generation.
- Zero burn.
- Negative or invalid burn inputs.
- Cash below the minimum threshold.
- Threshold breach during a forecast period.
- No breach within the forecast horizon.
- Missing receipts or payments.
- Delayed customer collections.
- Increased costs.
- Delayed funding.
- Restricted cash.
- Duplicate funding or cash-flow records.
- A scenario in which the forecast ends before a breach.
- Differences between reference runway and forecast-based runway.
- Consistency between displayed values, scenario assumptions and charts.

Verify that projected balances reconcile mathematically to the opening balance, dated inflows, outflows and eligible funding.

---

8. CROSS-FUNCTIONAL INTELLIGENCE AND ROOT-CAUSE ANALYSIS

BOSS must connect relevant information across the business rather than simply report separate observations.

Audit whether the existing intelligence engine can connect, for example:

- Declining cash.
- Increasing customer receivables.
- Positive accounting profitability.
- Rising payroll costs.
- No immediately identified compliance breach.

The system should explain, where supported by actual data, that positive profitability does not necessarily imply sufficient cash availability.

Slower collections and rising workforce costs may create liquidity pressure even when reported profit remains positive.

The system should investigate:

1. Whether the observations are based on consistent reporting periods.
2. Whether receivables are overdue or collections have slowed.
3. Whether payroll increases are material relative to revenue and cash generation.
4. Whether other cash outflows or working-capital movements contribute to the position.
5. How these factors affect projected liquidity.
6. Which factors have the greatest quantified impact.
7. What evidence is missing before a causal conclusion can be drawn.
8. Which actions could reasonably improve the forecast.

Potential actions may include accelerating overdue collections, reviewing discretionary expenditure or revising the cash forecast.

These are examples, not automatic recommendations.

Do not assert causation merely because two metrics move together.

Distinguish observed facts, correlations, plausible explanations, verified causes and forecasts.

Quantify the likely impact of an action only when the available data and calculation method support that estimate. Otherwise, state that the impact is unquantified or uncertain.

Avoid presenting the same underlying incident as several independent problems across Finance, Accounting, Auditing, Compliance and Risk.

Use shared issue identifiers or an equivalent traceable relationship where appropriate.

---

9. AI INTELLIGENCE BRIEF

Audit the existing BOSS Intelligence Brief and its supporting AI workflow.

The brief should provide a concise, evidence-backed executive interpretation of the current business position.

For each material finding, it should distinguish:

- What has happened or may happen.
- Supporting evidence.
- Relevant reporting period.
- Business impact.
- Root cause, if established.
- Forecast implication.
- Uncertainty and missing evidence.
- Recommended management response.

The intelligence brief should identify the most material issues rather than simply count exceptions.

Prioritisation should account for:

- Urgency.
- Materiality.
- Likelihood.
- Time sensitivity.
- Expected benefit.
- Confidence in the evidence.
- The consequences of delaying action.

Do not invent a numerical prioritisation formula if none exists. If a formal ranking model is required, document the proposed formula, explain the inputs and obtain approval before changing production behaviour.

AI-generated explanations must be traceable to the data and calculation outputs used to generate them.

The AI must not:

- Invent financial figures or business events.
- Fabricate supporting evidence.
- Treat missing information as proof of good health.
- Misrepresent forecasts as actual results.
- Invent causal relationships.
- Change deterministic calculations.
- Present an uncertain hypothesis as a confirmed fact.
- Claim that a recommended action will achieve an outcome without adequate support.

Where evidence is incomplete, explicitly disclose uncertainty.

Where an assessment cannot be calculated reliably, show an appropriate warning or unavailable state instead of producing a misleading score.

Use the organisation's actual data and configured business rules. Never substitute illustrative demo values for real production assessments.

---

10. PRIORITISED AI ACTION PLAN

Every recommendation should follow a consistent structure.

Required fields:

1. Recommendation ID.
2. Finding.
3. Evidence and supporting records.
4. Business impact.
5. Recommended action.
6. Priority and urgency.
7. Accountable owner.
8. Deadline.
9. Expected measurable outcome.
10. Current status.
11. Confidence or uncertainty.
12. Completion evidence.
13. Actual outcome and follow-up measurement.

Where applicable, also include dependencies, approval requirements, linked incidents and the date the recommendation was generated or last reviewed.

Recommendation priorities

Audit the existing priority model.

It should rank actions according to materiality, urgency, likelihood, time sensitivity and expected benefit, not simply the number of exceptions.

Criticality must be governed by explicit business rules.

An illustrative action plan might include:

- P1: Resolve a critical risk within 48 hours.
- P2: Protect cashflow within 7 days.
- P3: Investigate workforce efficiency within 30 days.

These priorities and deadlines are illustrative only. They must not be hardcoded as universal deadlines.

Actual deadlines should depend on verified evidence, severity, legal obligations, operational context, configured policies and authorised management decisions.

Accountability and outcomes

Audit whether BOSS supports:

- Assignment to a named, authorised owner or accountable role.
- Due dates.
- Pending, in-progress, completed and overdue statuses.
- Escalation rules where configured.
- Evidence of completion.
- Expected outcomes and measurable targets.

- Actual outcomes.
- Effectiveness reviews.
- Reopening actions that did not resolve the problem.
- An audit trail of status and ownership changes.

A recommendation must not be marked completed merely because someone changed its status.

Where feasible, track whether the intervention produced the expected improvement.

Human approval

Require appropriate authorised review for material decisions involving funding, significant spending, workforce changes, regulatory matters or other high-impact actions.

The AI may support decisions, but it must not independently execute consequential business actions unless an explicitly approved, secure workflow permits that action.

---

11. DATA QUALITY, CONFIDENCE AND FINANCIAL INTEGRITY

These are core safeguards, not optional dashboard enhancements.

11.1 Source data validation

Inspect whether BOSS validates:

- Data source and ownership.
- Reporting period.
- Data freshness.
- Completeness.
- Units and currency.
- Reconciliation status.
- Duplicate records.
- Conflicting records.
- Invalid or anomalous values.
- Relevant dependencies.
- Calculation eligibility.

Do not silently replace missing data with zero or treat missing evidence as a healthy result.

Define explicit behaviour for incomplete, stale or conflicting data.

11.2 Actuals versus estimates

Clearly distinguish:

- Verified actuals.
- Management forecasts.
- Modelled scenarios.
- AI-generated interpretations.
- Unverified or contingent assumptions.

Display relevant reporting dates, forecast horizons, data freshness and scenario assumptions.

11.3 Confidence

Keep data confidence separate from the health score.

Audit whether BOSS can explain what makes an assessment reliable or unreliable, including missing data, stale inputs, reconciliation failures and forecast uncertainty.

Do not invent a statistical confidence percentage unless a defensible calculation and validation method exists.

Distinguish data completeness, source reliability, forecast accuracy and AI interpretation uncertainty rather than collapsing unrelated concepts into one unsupported number.

11.4 Financial integrity

Inspect the calculation paths for:

- Currency and unit consistency.
- Period alignment.
- Duplicate transactions.
- Opening and closing balance reconciliation.
- Correct treatment of cash inflows and outflows.
- Forecast scenario consistency.
- Correct handling of restricted cash and funding.
- Precision and rounding.
- Missing or invalid values.
- Repeatability of calculations.

Use appropriate decimal or fixed-precision monetary representations supported by the existing technology stack.

Avoid introducing avoidable floating-point rounding errors into financial calculations.

---

12. DOUBLE-COUNTING, VERSIONING AND AUDIT TRAILS

12.1 Prevent double-counting

A single underlying incident may appear across several business dimensions.

Audit whether BOSS can link related observations to a common underlying issue, incident or source record.

Ensure that the same exposure is not treated as multiple independent incidents merely because several modules reference it.

Where the same event legitimately affects several dimensions, preserve the distinct dimension impacts while avoiding misleading duplication in overall risk aggregation and executive summaries.

12.2 Versioned calculations

Preserve enough information to reproduce historical results, including:

- Source values or permitted source references.
- Reporting period.
- Calculation formulas and version.
- Dimension weights.

- Score thresholds.
- Critical-risk rules and versions.
- Forecast assumptions.
- Scenario configuration.
- Relevant model and prompt versions.
- Calculation timestamp.
- Data freshness and reconciliation state.

Respect data-retention, access-control and privacy requirements.

Do not store sensitive information unnecessarily.

12.3 Reproducibility

Where the same permitted input data, configuration and rule version are used, deterministic calculations should reproduce the same result.

A later change to a weight, formula or rule must not silently rewrite historical assessment results.

Clearly distinguish a historical result from a recalculated result using newer rules.

12.4 Audit logging

Where applicable, preserve an auditable history of material changes to:

- Weights and thresholds.
- Critical-risk rules.
- Forecast assumptions.
- Recommendation ownership.
- Action status.
- Management approvals.
- Overrides and overrides' resolution.
- Calculation versions.

Ensure that audit logs are protected against unauthorised modification and do not expose secrets or unnecessary personal information.

---

13. FORECAST VALIDATION AND MODEL GOVERNANCE

Audit whether BOSS can evaluate the quality of its financial forecasts and management recommendations over time.

13.1 Historical back-testing

Where reliable historical data exists, compare prior assessments and forecasts with subsequent actual outcomes.

Evaluate relevant outcomes such as:

- Cash forecast errors.
- Actual versus forecast cash balances.
- Timing of liquidity threshold breaches.
- Missed payment warnings.

- Unanticipated control failures.
- Critical incidents.
- Action completion and effectiveness.

Avoid claiming predictive performance based only on illustrative examples or a handful of observations.

13.2 Forecast accuracy

Use appropriate measures for the type of forecast, the available data and the business context.

Document the calculation method and limitations.

Do not use an accuracy metric that becomes misleading when actual values are zero or close to zero.

Track forecast errors by horizon and scenario where appropriate.

13.3 Historical integrity

Do not retrospectively replace historical forecasts with later information without preserving the original version.

Record the original forecast, its assumptions, subsequent actual results and any later recalculation separately.

13.4 Stress testing

Where supported by data, test scenarios involving:

- Delayed customer collections.
- Reduced revenue.
- Margin deterioration.
- Increased payroll or operating costs.
- Unexpected expenditure.
- Funding delays or cancellation.
- Debt or statutory payment commitments.
- Significant operational incidents.
- Other organisation-specific material risks.

Use documented assumptions and show the effect on projected liquidity and the overall assessment.

13.5 Pilot and monitoring

Where operationally appropriate, compare BOSS recommendations with finance, risk and business leaders' assessments.

Investigate material disagreements and record the resulting adjustments to approved rules.

After launch, monitor:

- Forecast errors.
- Missed warnings.
- False alarms.
- Data-quality failures.
- Action completion rates.
- Recommendation effectiveness.
- Material model or configuration changes.

Do not claim that BOSS is independently validated, predicts business failure or has proven predictive accuracy unless suitable evidence supports that claim.

---

14. SECURITY, AUTHORISATION AND SAFE BUSINESS OPERATIONS

Audit the implementation of the intelligence features against the application's existing security architecture.

Inspect:

- Authentication and authorisation.
- Role-based access to financial and workforce information.
- Server-side permission enforcement.
- Protection of confidential records.
- API input validation.
- Safe handling of external AI outputs.
- Secret management.
- Error handling.
- Logging practices.
- Protection against unauthorised changes to business rules.
- Approval workflows for material actions.
- Appropriate separation of read-only analysis and write operations.

Do not expose sensitive source records in AI prompts, logs, reports or executive views unless authorised and necessary.

Do not allow an AI-generated recommendation to bypass server-side permissions or approved business rules.

Do not add new external services or transfer business data to a new AI provider without assessing the existing integration, data handling implications and required approval.

If an existing implementation presents a material security risk, document the evidence and recommend remediation. Do not conduct destructive security tests or access production systems without explicit authorisation.

---

15. ACCEPTANCE TESTS AND REGRESSION TESTING

Build an appropriate verification suite using the existing test framework and architecture.

Do not introduce an entirely new testing stack without justification and approval.

A. Executive experience

Verify that:

- The six intelligence layers are represented.
- Key figures come from the intended data sources.
- Reporting dates and freshness are clear.
- Drill-downs lead to relevant supporting information.
- Missing or stale data is visibly disclosed.

- The interface does not present illustrative values as actual organisational results.

B. Health model

Verify:

- Correct dimension weights.
- Correct weighted arithmetic.
- Normalisation and boundary conditions.
- Score-band behaviour.
- Critical-risk overrides.
- Missing-data handling.
- Multiple critical events.
- Historical reproducibility.
- Explanation consistency with calculated results.

C. Cashflow

Verify:

- Reference runway.
- Forecast-based runway.
- Minimum cash threshold.
- Dated receipts and payments.
- Scenario consistency.
- Funding treatment.
- Forecast breach dates.
- No-breach horizon handling.
- Positive cash generation.
- Zero or near-zero burn.
- Restricted cash.
- Reconciliation of projected balances.

D. Cross-functional reasoning

Verify that:

- Relevant observations can be linked across dimensions.
- Causal claims are not invented.
- Evidence is accessible.
- Duplicate incidents are not misleadingly counted more than once.
- Forecast impacts are reproducible where quantified.
- Uncertainty is disclosed.

E. Recommendations and accountability

Verify:

- Recommendation fields are complete where required.
- Priorities reflect configured rules.
- Owners and deadlines are handled correctly.
- Authorisation is enforced.

- Status transitions are valid.
- Completion evidence is captured.
- Outcomes can be evaluated.
- Overdue and unresolved critical actions are handled correctly.

F. Data integrity and governance

Verify:

- Missing and stale inputs.
- Duplicate records.
- Conflicting sources.
- Reconciliation failures.
- Calculation versioning.
- Audit logs.
- Historical result preservation.
- Permission enforcement.
- Reproducible calculations.

G. Existing application regression

Verify that the changes do not unintentionally break existing:

- User journeys.
- Authentication.
- Roles and permissions.
- APIs and integrations.
- Database relationships.
- Reporting workflows.
- Other business modules.
- Deployment and configuration.

Run relevant unit, integration and end-to-end tests where available and safe.

Run appropriate linting, type checking and build commands.

Do not report a test as passed if it was not executed or if its result was unsuccessful.

If automated coverage is missing, identify the required tests and provide the strongest safe alternative verification available.

---

16. FINAL EXECUTIVE SCREEN AND DESIGN ACCEPTANCE

The recommended executive screen should make three questions answerable within seconds.

Where do we stand?

Overall health, business status, cash position, runway and major changes.

What needs attention?

Critical risks, declining dimensions, evidence-backed root causes, uncertainty and forecast exposure.

What must we do?

Prioritised actions, accountable owners, deadlines, expected impact and outcome tracking.

The interface should support:

- A consolidated executive overview.
- Clear distinction between score and final status.
- Prominent critical-risk alerts.
- Both reference and forecast-based cash runway.
- Optimistic, base and downside scenarios.
- Six-dimension health summaries.
- Evidence-linked AI explanations.
- Prioritised actions.
- Confidence and data-quality disclosures.
- Supporting drill-downs.

Do not hardcode example scores, cash balances, forecast values, owners, deadlines or risks.

The application must use the organisation's actual data, approved configurations and supported calculation logic.

---

17. REQUIREMENTS PRIORITY AND IMPLEMENTATION ORDER

During the audit, assign priority based on actual risk and dependencies.

Use these guidelines:

P0 - Critical integrity or safety

Examples include calculations that materially misrepresent financial position, missing protections around critical-risk overrides, unauthorised access, exposure of sensitive data, or dangerous data-integrity failures.

P1 - Core decision-support correctness

Examples include material errors in health scoring, cash forecasting, data reconciliation, evidence traceability or prioritisation.

P2 - Functional completeness

Examples include missing scenario comparisons, action ownership, outcome tracking, supporting drill-downs or incomplete integration between dimensions.

P3 - Improvements

Examples include presentation refinements, usability enhancements and non-critical workflow optimisations.

These are prioritisation guidelines, not a reason to ignore lower-priority requirements.

A material regulatory deadline, security concern or other urgent operational condition may require a higher priority than its category suggests.

Explain the reason for each assigned priority.

Resolve dependencies before implementing downstream functionality.

Do not introduce unnecessary architectural changes merely to satisfy a cosmetic preference.

---

18. IMPLEMENTATION RULES AFTER APPROVAL

Once I approve the implementation plan:

1. Recheck Git status and preserve existing user changes.
2. Work from the approved requirements register.
3. Implement related requirements in small, controlled groups.
4. Review existing implementations before creating new components or services.
5. Avoid unnecessary dependency changes.
6. Keep changes focused and reviewable.
7. Add or update tests alongside each relevant implementation.
8. Update the requirements register with implementation evidence and verification results.
9. Run relevant tests after each coherent group of changes.
10. Investigate failures instead of suppressing them or weakening tests.
11. Review the diff for accidental modifications.
12. Document database, environment and deployment changes separately.
13. Obtain approval before destructive operations, irreversible migrations, production changes or changes with significant data-loss risk.
14. Maintain a clear rollback plan for changes that could affect existing operation.
15. Do not silently skip blocked or ambiguous requirements.

Do not automatically mark the entire application complete when implementation work finishes.

Implementation and verification are separate milestones.

---

19. FINAL AUDIT AND REQUIRED DELIVERABLES

After the approved changes and testing are complete, perform a final reconciliation against the original requirements in this master prompt.

Do not rely exclusively on the previous audit report. Inspect the current source code and actual test evidence.

Deliver the following:

Deliverable 1 - Executive audit summary

Summarise the existing application's architecture, principal strengths, material gaps, critical findings and recommended next steps.

Deliverable 2 - Complete requirements traceability register

Include every requirement ID, acceptance criteria, initial finding, implementation location, final status, verification evidence and outstanding work.

Deliverable 3 - Original versus final comparison

For each requirement, show:

- Original state.
- Approved change.
- Final implementation.

- Test evidence.
- Remaining limitations.

Deliverable 4 - Files changed

List files created, modified and deleted, with reasons.

Identify changes to dependencies, schemas, migrations, configuration, APIs and integrations.

Deliverable 5 - Test report

Report the exact relevant commands executed and the actual results.

Include failures, skipped checks, environment limitations and unresolved issues.

Deliverable 6 - Financial model verification

Document:

- Score formulas and configured weights.
- Score thresholds.
- Critical-risk rules.
- Cash runway formulas.
- Forecast assumptions.
- Scenario calculations.
- Data-quality controls.
- Versioning and reproducibility.
- Known limitations.

Distinguish tested mathematical correctness from validation against real organisational outcomes.

Deliverable 7 - AI intelligence verification

Show whether recommendations are grounded in supporting evidence, whether uncertainty is disclosed, whether cross-functional relationships are justified, and whether actions have measurable outcomes.

Deliverable 8 - Regression and security findings

Document material regressions, permission issues, data-integrity risks, security concerns and compatibility limitations.

Deliverable 9 - Remaining work

List all blocked, partially implemented, unverified, failed and not-applicable requirements with explanations.

Do not hide unresolved issues behind a high overall completion percentage.

Deliverable 10 - Final acceptance decision

State whether the application is:

- NOT READY FOR ACCEPTANCE.
- READY FOR REVIEW WITH KNOWN LIMITATIONS.
- READY FOR ACCEPTANCE AGAINST THE DEFINED AND VERIFIED REQUIREMENTS.

Use the final category only when every mandatory requirement has been accounted for, all required verification has passed, and no unresolved critical issue prevents acceptance.

This is not a certification of regulatory compliance, independent model validation or production safety.

---

20. FINAL OPERATING INSTRUCTIONS

Start with Phase 0 and Phase 1.

Then audit the complete requirements in this document.

Do not modify the application during the initial audit.

Do not create the traceability file until I approve the initial audit report.

Do not implement changes until I approve the proposed implementation plan.

If the application is too large to inspect in one pass, divide the work into modules and maintain a master register so that no requirement disappears between sessions.

If the session ends before the work is complete, record completed inspections, outstanding requirements, evidence collected and the exact next steps in the approved project tracking document.

Never substitute a general summary for the requirement-by-requirement audit.

Never assume that a feature is missing without checking the relevant implementation.

Never assume that a feature works merely because it exists.

Never invent results, data, evidence or test outcomes.

Never rebuild the existing application unnecessarily.

Your immediate task is to inspect the project, establish the actual technology stack and produce the initial read-only audit plan. Then conduct the complete audit and report your findings. STOP and wait for my approval before making any changes.

-----^-----